



Anti-Gravital Framework

El framework web del siguiente cuarto de siglo. Construido en Rust puro. Desde Panamá hacia el mundo.

DOCUMENTO

Blueprint v4.0 · Documento Maestro

ORIGEN

República de Panamá

LICENCIA

Apache 2.0

FECHA

Mayo 2026

Tabla de contenidos

PARTE I — CONTEXTO

1. Resumen ejecutivo	5
2. Manifiesto y posicionamiento	7
3. Qué es Anti-Gravital y qué no es	9
4. Análisis del estado del arte	12

PARTE II — ARQUITECTURA

5. Arquitectura del ecosistema	17
6. Núcleo: Shield y Core	22
7. Anti-DSL — especificación	28

PARTE III — MÓDULOS DEL ECOSISTEMA

8. Módulos batteries-included	35
9. Sistema de plugins WASI	42
10. ag-cloud: despliegue simplificado	46
11. ag-ai y Knowledge Graph	50
12. ag-migrate: importadores	54
13. ag-mobile: Flutter bridge	57

PARTE IV — CALIDAD Y OPERACIÓN

14. Observabilidad	61
15. Modelo de seguridad	64
16. Objetivos de rendimiento	67

PARTE V — PROYECTO

17. Modelo de gobernanza Open Source	71
18. Análisis de riesgos	74

19. Hoja de ruta y puertas de verificación	77
.....	
20. Estrategia go-to-market	92
.....	

APÉNDICES

A. Glosario técnico	95
.....	
B. Comparativa de mercado	98
.....	

PARTE I

Visión, alcance y contexto

Esta primera parte establece qué es Anti-Gravital, qué no es, por qué existe y dónde se ubica en el mercado actual de frameworks. Es la base conceptual sobre la que se apoyan las decisiones técnicas de las partes siguientes.

1. Resumen ejecutivo

Anti-Gravital es un ecosistema de software libre para construir aplicaciones backend de alto rendimiento, escrito en Rust puro, con tres propiedades fundamentales que lo distinguen del resto del mercado de frameworks web actuales.

La primera es la **ausencia total de runtime externo**: el resultado de un proyecto Anti-Gravital es un binario estático autocontenido que se ejecuta sobre el sistema operativo sin intérprete ni máquina virtual de por medio. Esto elimina la JVM, CPython, Node.js y CLR del path de ejecución y, con ellos, las pausas de recolección de basura, los segundos de arranque en frío y los cientos de megabytes de memoria base que esos runtimes consumen antes de procesar la primera petición.

La segunda es el enfoque **schema-first** apoyado en un lenguaje de definición de dominio llamado Anti-DSL, archivos con extensión `.ag`. Un único archivo describe modelos, endpoints, políticas, validaciones, errores y relaciones; el compilador del DSL deriva de allí el código Rust, los clientes tipados para frontend y aplicaciones móviles, la documentación OpenAPI, y las migraciones de base de datos. El contrato es una sola fuente de verdad, y la deriva de esquema (*schema drift*) deja de ser una clase de problema posible por construcción.

La tercera es una **arquitectura modular pensada como ecosistema**, no como framework monolítico. El núcleo es deliberadamente pequeño y se compone con módulos publicados de forma independiente. Cada módulo (`ag-auth`, `ag-data`, `ag-realtime`, `ag-cache`, `ag-storage`, `ag-observe`, `ag-cloud`, `ag-ai`, `ag-mobile`, `ag-migrate`) tiene un dominio propio, versionado propio, y puede usarse de forma aislada en cualquier proyecto Rust. Esta separación hace al ecosistema sostenible a escala de comunidad y elimina el síndrome del «framework que intenta resolverlo todo».

TESIS DEL PROYECTO

El rendimiento de sistemas y la productividad del desarrollador no son fuerzas opuestas, sino problemas de diseño. Un framework correctamente diseñado puede ofrecer ambos simultáneamente sin compromisos ocultos. Anti-Gravital se construye sobre esa premisa.

El proyecto nace desde Panamá, con documentación bilingüe español/inglés desde el día cero. El primer foco de adopción es Latinoamérica; el horizonte es global. La licencia Apache 2.0 garantiza que no existirá nunca una versión Enterprise cerrada ni features reservadas para clientes pagos: la totalidad del ecosistema es y será código abierto.

Este documento describe en detalle cada componente, cada decisión arquitectónica, y los compromisos técnicos que sustentan el proyecto. El documento complementario *Hoja de Ruta y Puertas de Verificación* define la secuencia temporal de entregables y los criterios de bloqueo entre fases.

15

CRATES
INDEPENDIENTES

11

FASES CON PUERTAS
BLOQUEANTES

24-28

MESES HASTA V1.0
ESTABLE

100%

APACHE 2.0, SIN
ENTERPRISE CERRADA

2. Manifiesto y posicionamiento

Durante los últimos veinte años, la industria del software ha aceptado un compromiso que ya no es necesario: elegir entre rendimiento y productividad. Los frameworks más adoptados del mundo prosperaron resolviendo solo uno de los dos extremos. Spring Boot y .NET impusieron estructura empresarial al precio de máquinas virtuales pesadas y arranques de varios segundos. Django y FastAPI hicieron posible que equipos pequeños construyeran APIs en horas, al precio del GIL y un intérprete que pone un techo invisible al rendimiento. Node.js trajo desarrollo isomórfico al precio de un event loop monohilo y un ecosistema npm crónicamente vulnerable a ataques de cadena de suministro.

Ninguno de estos frameworks es malo. Todos resuelven problemas reales. Pero todos fueron diseñados en una época anterior a tres fenómenos convergentes que cambian el cálculo: la madurez de producción del ecosistema Rust, la llegada de agentes de IA capaces de escribir código de calidad a velocidades sobrehumanas, y el desencanto de la industria con la complejidad operacional de los stacks multilenguaje.

El nombre del proyecto describe la tesis. Los frameworks actuales tienen *gravedad*: te atan a intérpretes, máquinas virtuales, runtimes externos y capas de abstracción que cobran en latencia, memoria y complejidad operacional. Anti-Gravital rompe con esa gravedad desde los cimientos: sin JVM, sin GC, sin intérprete, sin runtime externo. Solo código máquina nativo, seguridad de memoria garantizada en compilación, y concurrencia masiva sin costo de recolección de basura.

2.1 Posicionamiento explícito

Anti-Gravital no se posiciona contra ningún lenguaje ni ningún framework. Se posiciona como la capa backend y runtime unificada moderna para aplicaciones que necesitan tres cosas simultáneamente: rendimiento de sistemas, productividad de framework de alto nivel, y simplicidad operacional de despliegue. El público objetivo no es el equipo que ya tiene un stack Spring funcionando en producción y no tiene dolor — es el equipo que está empezando un proyecto nuevo, o que ha alcanzado los límites estructurales de Python/Node.js, o que necesita reducir la huella de memoria de su flota de servicios.

La estrategia de adopción se construye sobre **interoperabilidad y migración gradual**, no sobre el reemplazo agresivo de stacks existentes. Los importadores oficiales (OpenAPI, Prisma, Sequelize, Django ORM, modelos FastAPI/Pydantic) son ciudadanos de primera clase, no un afterthought.

2.2 Los tres fenómenos convergentes

La madurez del ecosistema Rust (2022–2026). Tokio, Axum, Tower, SQLx y las macros procedurales alcanzaron estabilidad de producción. Cloudflare, Discord y AWS ejecutan Rust en cargas críticas. La pregunta ya no es «¿es Rust maduro?» sino «¿qué falta para que sea productivo en aplicaciones de negocio?». La respuesta es un framework que resuelva el boilerplate sin sacrificar las garantías del lenguaje.

La era de los agentes de IA (2024–2026). Claude, GitHub Copilot y Cursor pueden generar código de calidad a velocidades que superan ampliamente la velocidad de tipeo humana. El cuello de botella ya no es «puedo escribir el código» sino «tengo un contrato claro y verificable que guíe lo que se escribe». Anti-DSL es ese contrato, y el compilador de Rust actúa como segundo revisor que rechaza en build-time el código incorrecto que el agente pudo haber producido.

El desencanto con la complejidad (2023–2026). La industria regresa a monolitos bien contruidos. Los equipos quieren un stack simple de operar, fácil de depurar y predecible bajo carga. Un binario estático sin dependencias de runtime es la respuesta más simple posible al problema operacional.

3. Qué es Anti-Gravital y qué no es

La definición clara del alcance es probablemente la decisión arquitectónica más importante del proyecto. Un framework que intenta ser todo termina siendo nada. Esta sección establece los límites explícitos.

3.1 Qué es Anti-Gravital

- ✓ Un **runtime backend Rust** de alto rendimiento para servicios HTTP, WebSocket y SSE.
- ✓ Un **lenguaje de definición de dominio** (Anti-DSL, archivos `.ag`) y su compilador.
- ✓ Una **CLI unificada** (`ag`) para creación, generación, desarrollo, build, despliegue y administración.
- ✓ Un **conjunto de módulos opcionales** publicados como crates Rust independientes (auth, data, realtime, cache, storage, observe).
- ✓ Un **sistema de plugins WASI** para extensibilidad multilenguaje aislada.
- ✓ Una **capa de orquestación de despliegue** simplificada al estilo Railway/Fly.io para casos comunes (no un reemplazo de Kubernetes).
- ✓ Un **generador de SDKs tipados** para TypeScript, Dart y otros lenguajes cliente.

- ✓ Un **conjunto de importadores de migración** desde frameworks legacy (OpenAPI, Prisma, Django, FastAPI, Sequelize, GraphQL).
- ✓ Un **knowledge graph** auto-generado que mantiene la documentación arquitectónica sincronizada con el código.

3.2 Qué NO es Anti-Gravital

Esta lista es igualmente importante. Establece límites explícitos que protegen al proyecto de crecer en direcciones inmanejables. Anti-Gravital **no** intenta ni intentará lo siguiente:

No reemplaza Kubernetes. Para cargas que justifican Kubernetes, Anti-Gravital se despliega *sobre* Kubernetes como cualquier otro binario contenedorizado. El módulo `ag-cloud` cubre el rango Docker Compose hasta Fly.io. Cuando un equipo necesita orquestación a escala de cientos de nodos, usa Kubernetes y se acabó.

No reemplaza Flutter ni React Native. Anti-Gravital no es un framework de UI multiplataforma. Es el backend nativo ideal *para* aplicaciones Flutter y React Native, con generación automática de SDKs cliente tipados, autenticación nativa, realtime, offline sync y streaming. La intuición correcta es: Flutter resuelve la capa de presentación cross-platform; Anti-Gravital resuelve la capa de servicios sobre la que Flutter consume datos.

No reemplaza React, Vue, Svelte ni Next.js. El módulo `ag-ui` ofrece SSR + HTMX para casos donde un stack JS completo es excesivo, pero no compite con frameworks frontend establecidos. Para aplicaciones SPA o SSR ricas, el patrón recomendado es Anti-Gravital como backend + Next.js (o equivalente) como frontend, comunicándose vía cliente TypeScript generado automáticamente desde el schema `.ag`.

No reemplaza Docker. Genera Dockerfiles. Se ejecuta en contenedores. No reinventa el formato OCI ni intenta sustituir las herramientas de contenedorización del ecosistema.

No reemplaza PostgreSQL, Redis, MinIO ni NATS. Se integra con ellos como dependencias externas estándar. `ag-data` usa PostgreSQL/MySQL/SQLite vía SQLx; `ag-cache` usa Redis o moka en memoria; `ag-storage` usa S3 o MinIO; `ag-realtime` usa NATS embebido o cluster externo.

No reemplaza Terraform ni Pulumi. El módulo `ag-cloud` orquesta despliegues simples; para infraestructura compleja multi-cloud con políticas, IaC declarativa y módulos compartidos, Terraform sigue siendo la herramienta correcta.

No es un motor de juegos, ni un framework de cómputo científico, ni una alternativa a Unreal Engine, Unity, NumPy, PyTorch o TensorFlow. Estos dominios tienen herramientas especializadas que Anti-Gravital no intenta replicar.

3.3 La regla de interoperabilidad

Cuando exista una herramienta dominante en un dominio adyacente, la estrategia es **integrar, no reemplazar**. Esta regla evita que el proyecto crezca en direcciones inmanejables y mantiene el alcance defendible frente a terceros que evalúan adopción.

4. Análisis del estado del arte

Esta sección documenta el contexto competitivo en términos técnicos, sin retórica adversarial. Cada framework analizado resuelve un conjunto real de problemas; el análisis identifica las limitaciones estructurales que Anti-Gravital pretende abordar.

4.1 Spring Boot y el ecosistema JVM

Spring Boot domina el desarrollo empresarial Java y Kotlin con dos décadas de ecosistema maduro. Sus debilidades estructurales derivan de la JVM: un consumo de memoria base de 256–512 MB antes de servir el primer request, tiempos de arranque de 6–8 segundos incompatibles con cómputo serverless, y verbosidad de configuración que añade complejidad mantenible. GraalVM Native Image mitiga parcialmente el arranque y la memoria, pero introduce sus propias limitaciones (reflexión limitada, compatibilidad incompleta de librerías, tiempos de compilación largos). El compromiso fundamental — un runtime gestionado con GC — permanece.

4.2 ASP.NET Core y .NET

Técnicamente uno de los frameworks gestionados más rápidos del mercado, con C# moderno y expresivo. El CLR con GC mantiene pausas medibles en p99 bajo carga sostenida. La dirección técnica del ecosistema es unilateral de Microsoft. La seguridad de memoria no está garantizada por el compilador; los bugs de race conditions y null reference exceptions son posibles. La adopción fuera del ecosistema Microsoft sigue siendo limitada por razones culturales más que técnicas.

4.3 Django y FastAPI

Django mantiene la mejor experiencia de prototipado del mundo Python, con un ecosistema rico para administración, autenticación y plantillas. FastAPI elevó el estándar de DX en APIs Python con tipos Pydantic, generación automática de OpenAPI y soporte async nativo. Ambos comparten el techo estructural de CPython: el Global Interpreter Lock impide concurrencia real CPU-bound dentro de un proceso, lo que obliga a escalar con múltiples procesos (Gunicorn, Uvicorn workers) multiplicando el consumo de memoria. El soporte async de Django sigue siendo parcial; muchas librerías del ecosistema permanecen sincrónicas.

4.4 Node.js, Express y NestJS

Node.js trajo JavaScript al servidor y el ecosistema npm es el más amplio de la industria. El event loop monohilo de V8 es óptimo para I/O concurrente pero se degrada con cualquier trabajo CPU-bound. La cadena de suministro npm es crónicamente vulnerable: la dependencia transitiva media de un proyecto Node.js moderno excede las 200 librerías, y los incidentes de paquetes comprometidos son recurrentes. TypeScript añade seguridad de tipos en desarrollo, pero en runtime sigue siendo JavaScript.

4.5 Next.js y los frameworks fullstack JS

Next.js representa la convergencia frontend/backend en JavaScript. Server Components y Server Actions reducen el boilerplate de APIs internas. Las debilidades estructurales son herencia de Node.js: cold starts en serverless, acoplamiento de facto con Vercel, inadecuación para WebSockets persistentes, estado compartido y procesamiento de larga duración. Next.js es una excelente capa de presentación; no es un backend robusto para cargas críticas.

4.6 Axum, Actix-Web y Rocket (Rust)

Los frameworks Rust actuales son técnicamente excelentes en rendimiento (top 10 de TechEmpower de forma consistente) pero ofrecen lo que la comunidad llama una experiencia *low-level*: el desarrollador construye desde cero la autenticación, la capa de datos, la observabilidad, la generación de clientes y el sistema de migraciones. Anti-Gravital se construye **sobre** Axum, Tokio y Tower como dependencias internas — no compite con ellos, sino que los empaqueta en una experiencia de framework completo con DSL, CLI y módulos opinados.

4.7 Conclusión del análisis

Existe un espacio de mercado real: un framework Rust enterprise-grade dominante todavía no existe. Spring Boot paga el costo histórico de la JVM. Node.js tiene límites estructurales de event loop. Python tiene problemas de concurrencia. Go sacrifica el sistema de tipos. Rust tiene runtime y rendimiento, pero le falta una experiencia de framework completa. Anti-Gravital pretende llenar ese hueco con coherencia arquitectónica, no con marketing adversarial.

Arquitectura del ecosistema y del núcleo

Esta parte describe la organización modular del proyecto, la arquitectura interna de `ag-core` con sus capas Shield y Core, y la especificación del lenguaje Anti-DSL con su estrategia de implementación incremental.

5. Arquitectura del ecosistema

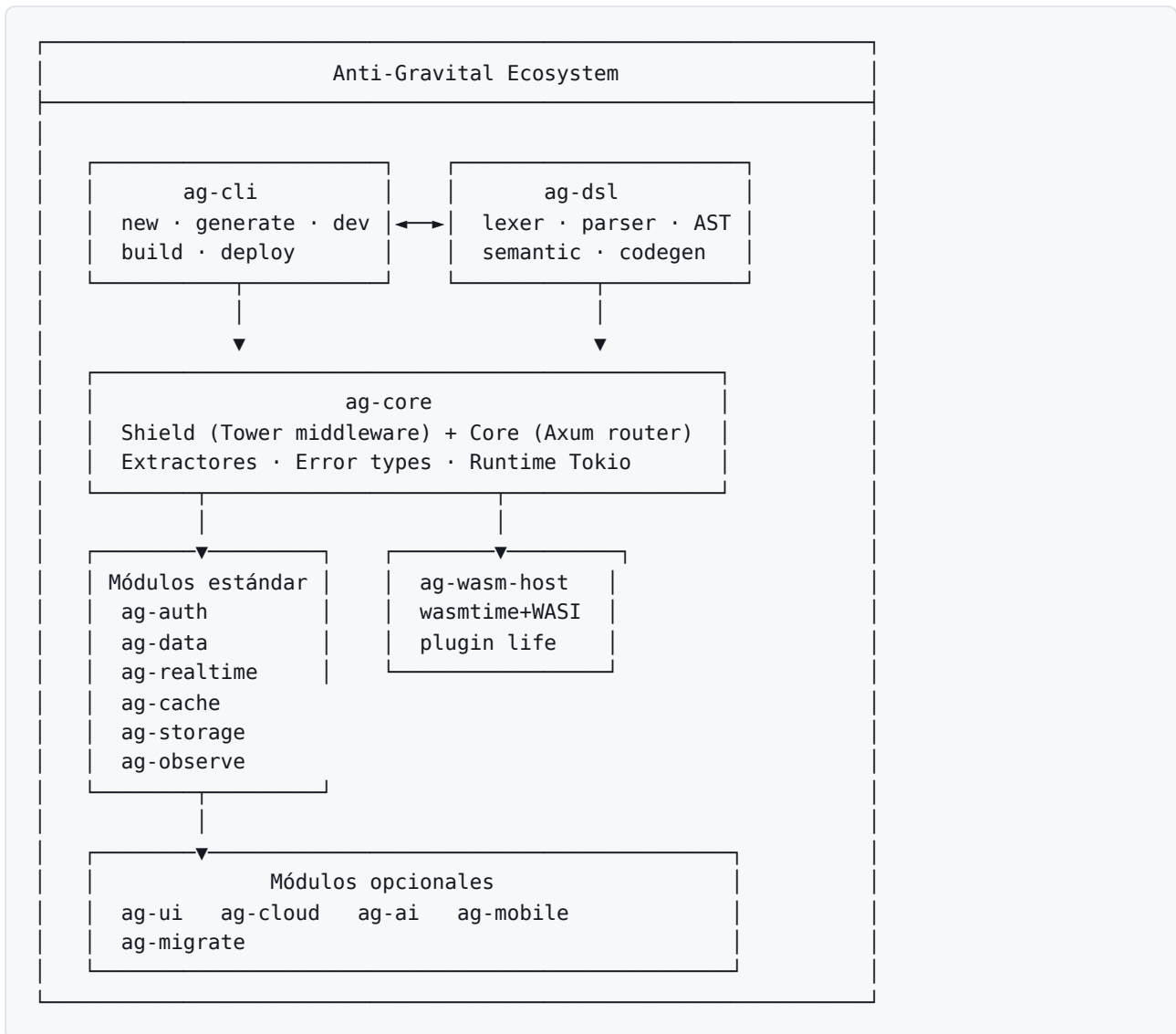
La decisión arquitectónica más importante derivada del análisis crítico del v3.0 fue **separar el núcleo de los ecosistemas**. El v3.0 intentaba ser simultáneamente framework backend, motor SSR, plataforma DevOps, orquestador AI, capa de observabilidad, framework móvil y sistema de plugins. Esto es inmanejable. La v4.0 reorganiza el proyecto como un ecosistema de crates Rust independientes, cada uno con un dominio propio, un mantenedor responsable, versionado semántico independiente y una superficie de API mínima.

5.1 Mapa del ecosistema

Crate	Dominio	Criticidad
<code>ag-core</code>	Runtime HTTP, router, extractores, error types, Shield/Core	Núcleo
<code>ag-dsl</code>	Lexer, parser, AST, análisis semántico y codegen del Anti-DSL	Núcleo
<code>ag-cli</code>	Binario <code>ag</code> : new, generate, dev, build, deploy, migrate	Núcleo
<code>ag-wasm-host</code>	Runtime de plugins WASI sobre wasmtime	Núcleo
<code>ag-auth</code>	WebAuthn, JWT Ed25519, OAuth2, RBAC, rate limiting	Estándar
<code>ag-data</code>	sqlx con verificación compile-time, migraciones, ORM tipado	Estándar
<code>ag-realtime</code>	WebSocket, SSE, NATS embebido, pub/sub	Estándar
<code>ag-cache</code>	moka en memoria, adaptador Redis, invalidación por evento	Estándar
<code>ag-storage</code>	S3, MinIO, filesystem local, URLs firmadas, procesamiento imagen	Estándar
<code>ag-observe</code>	tracing, OpenTelemetry, Prometheus, dashboards Grafana	Estándar
<code>ag-ui</code>	SSR con askama, hidratación selectiva, integración HTMX	Opcional
<code>ag-cloud</code>	Orquestación de despliegue Railway-like, Dockerfile gen	Opcional
<code>ag-ai</code>	Doc generation, schema suggestions, knowledge graph	Opcional
<code>ag-mobile</code>	Generación SDK Dart, auth nativo Flutter, offline sync	Opcional
<code>ag-migrate</code>	Importadores OpenAPI, Prisma, Django, FastAPI, Sequelize	Opcional

La distinción entre **núcleo**, **estándar** y **opcional** es importante. El núcleo es el conjunto mínimo que define lo que es Anti-Gravital. Los módulos estándar cubren el 90 % de las necesidades de producción de cualquier servicio backend y se instalan por defecto en los templates oficiales. Los módulos opcionales se añaden cuando el proyecto los necesita.

5.2 Diagrama del ecosistema



5.3 Reglas de dependencia entre crates

Para mantener el ecosistema sano se aplican reglas estrictas de dependencia que se verifican en cada pull request mediante un job de CI dedicado.

Primera regla: `ag-core` no depende de ningún otro crate del ecosistema Anti-Gravital. Es la base sobre la que todo lo demás se construye.

Segunda regla: los módulos estándar pueden depender de `ag-core` y de otros módulos estándar siempre que no haya ciclos. Por ejemplo, `ag-auth` puede depender de `ag-data` para persistencia de sesiones, pero `ag-data` no puede depender de `ag-auth`.

Tercera regla: los módulos opcionales pueden depender de cualquier crate núcleo o estándar. No pueden depender entre sí salvo casos explícitamente justificados (por ejemplo, `ag-mobile` puede depender de `ag-ai` para generación de código asistida).

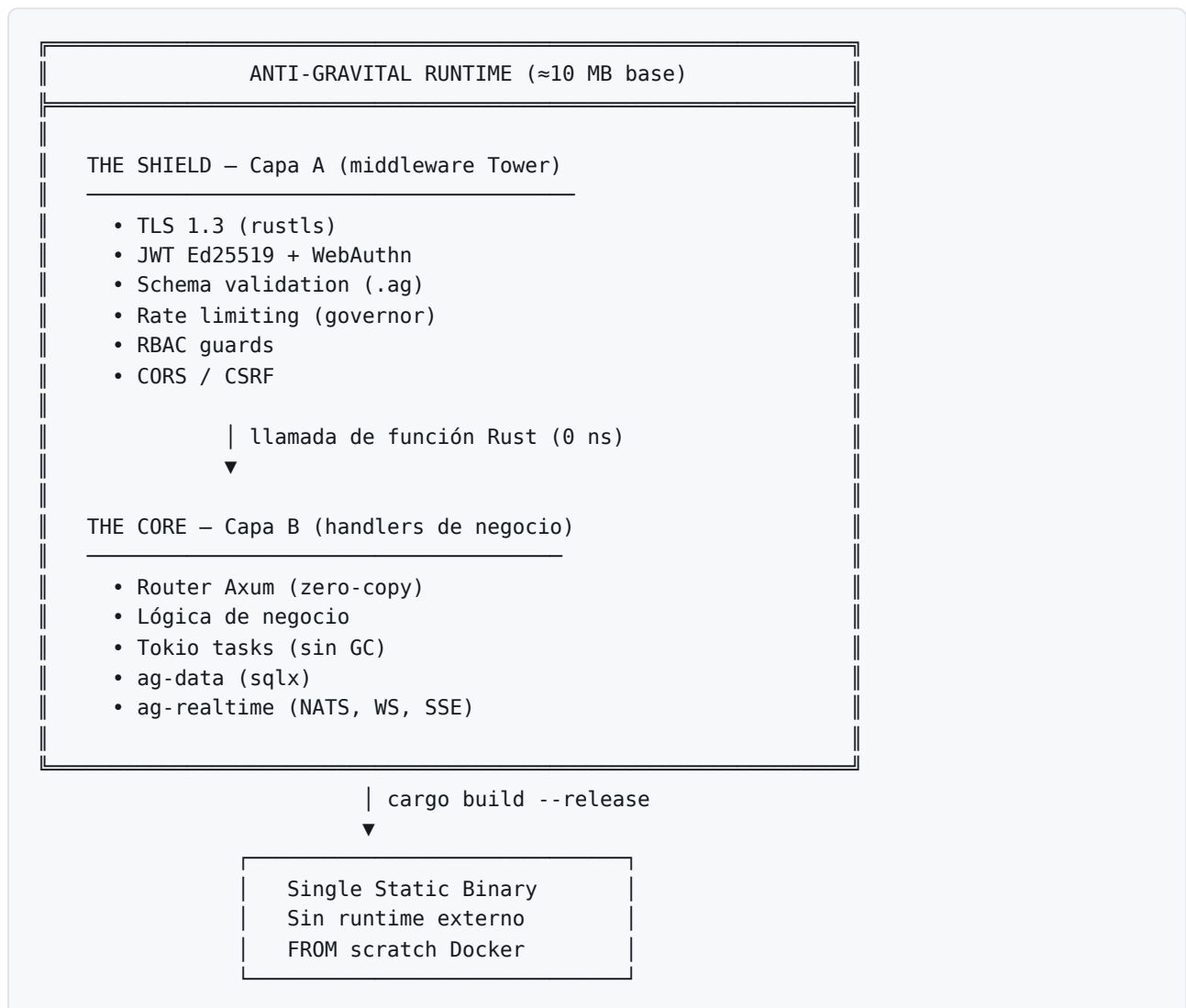
Cuarta regla: `ag-cli` depende de todos los demás crates a través de *features* de Cargo, de modo que el binario `ag` puede compilarse con un subconjunto reducido para entornos restringidos.

Quinta regla: todos los crates publican versiones semánticas independientes. Una breaking change en `ag-cache` no fuerza a `ag-core` a subir mayor. Esto es esencial para la sostenibilidad de un proyecto open source.

6. Arquitectura del núcleo: Shield y Core

`ag-core` es el crate fundamental del proyecto. Internamente está organizado en dos capas conceptuales — **Shield** y **Core** — que viven en el mismo proceso Rust. La comunicación entre capas es una llamada de función ordinaria: cero overhead de IPC, cero costo de serialización, un único stack trace de extremo a extremo.

6.1 Modelo de capas



6.2 The Shield

Todo lo que toca la red antes de que el request sea confiable. The Shield es una pipeline de Tower layers — el mismo modelo composable que usa Axum internamente. Stack técnico: `tokio` (runtime async M:N), `tower` (middleware composable), `rustls` (TLS 1.3 sin OpenSSL), `serde` / `serde_json` (zero-copy), `ring` (criptografía), `governor` (rate limiting con token bucket thread-safe).

El orden de las capas del Shield es deliberado: TLS termina primero, luego se aplica rate limiting (para rechazar abuso antes de gastar ciclos en autenticación), luego CORS, luego autenticación, luego autorización (RBAC), y finalmente validación del cuerpo del request contra el schema `.ag`. Un request que llega al Core ya es estructuralmente válido, autenticado, autorizado y dentro de cuota.

6.3 The Core

La lógica de negocio. El 80 % del código de la aplicación vive aquí, en handlers Rust generados desde el schema `.ag`. El desarrollador escribe únicamente el cuerpo de los handlers — las firmas, validaciones, extractores y tipos son generados por `ag generate`.

```
// Generado por `ag generate` – el developer rellena solo el cuerpo
pub async fn create_user(
    State(state): State<AppState>,
    ValidatedBody(req): ValidatedBody<CreateUserRequest>, // The Shield validó
    Claims(claims): Claims<AuthClaims>,                  // JWT ya verificado
) -> Result<Json<User>, AgError> {
    // Lo único que escribe el desarrollador:
    let user = state.db.users()
        .create(CreateUserParams {
            email: req.email,
            name: req.name,
        })
        .await?;

    state.events.emit("user.created", &user).await?;
    Ok(Json(user))
}
```

7. El lenguaje Anti-DSL

El Anti-DSL es el contrato único de un proyecto Anti-Gravital. Un archivo `.ag` describe modelos, endpoints, validaciones, errores, eventos y políticas; el compilador deriva todo el resto del código.

7.1 Ejemplo de schema

```
# schema.ag – La única fuente de verdad
model User {
  id      UUID      @primary @auto
  email   String    @unique  @max(255)
  name    String    @max(100)
  role    UserRole   @default(USER)
  created Timestamp @auto
}

endpoint CreateUser {
  method  POST
  path    /users
  auth    required
  policy  "user.role != BANNED"
  body    CreateUserRequest
  response User
  errors  [EmailTaken, ValidationError]
}

request CreateUserRequest {
  email String @email
  name  String @min(2) @max(100)
}
```

7.2 Artefactos generados

Desde este único archivo, el comando `ag generate` produce los siguientes artefactos coherentes entre sí:

`src/models.rs` con structs Rust + serde + validadores; `src/validators.rs` con la lógica de validación para The Shield; `src/handlers/stubs.rs` con las firmas de los handlers, donde el desarrollador rellena solo el cuerpo; `src/db/queries.rs` con queries sqlx verificadas en compile time contra la base de datos real; `src/db/migrations/` con las migraciones SQL versionadas; `ts/types.ts` con los tipos TypeScript para frontend; `ts/client.ts` con el cliente HTTP type-safe; `dart/models.dart` y `dart/client.dart` con el SDK para Flutter; y `openapi.yaml` con la documentación OpenAPI 3.1 actualizada.

GARANTÍA DE COHERENCIA

Un cambio en `schema.ag` → `ag generate` → todos los artefactos actualizados de manera atómica. El *schema drift* queda eliminado por construcción, no por disciplina humana.

7.3 Estrategia de implementación incremental del DSL

Un compilador completo (lexer, parser, análisis semántico, generadores de código, LSP, linter, formateador, sistema de versionado de gramática) es casi tan complejo como un compilador de lenguaje propósito general. Por eso el DSL se implementa de manera estrictamente incremental, con una capacidad mínima por iteración.

Versión	Capacidad del DSL	Hito asociado
v0.1	Modelos (campos, tipos primitivos, anotaciones básicas)	Fin Fase 3.1
v0.2	Endpoints (método, path, body, response)	Fin Fase 3.2
v0.3	Validaciones declarativas (@email, @min, @max, regex)	Fin Fase 3.3
v0.4	Relaciones entre modelos (1:1, 1:N, N:N)	Fin Fase 3.4
v0.5	Auth y políticas declarativas	Fin Fase 5
v0.6	Eventos y suscripciones	Fin Fase 6
v0.7	Plugin hooks (lifecycle, decoradores)	Fin Fase 9
v1.0	Gramática estable, congelada bajo semver	Fin Fase 10

Esta estrategia garantiza que cada versión del DSL es funcional y útil por sí misma, y permite que la comunidad empiece a construir sobre el DSL antes de que la gramática esté completa. La congelación de la gramática en v1.0 — momento a partir del cual los cambios incompatibles requieren un proceso de RFC formal — es uno de los hitos más importantes del proyecto.

PARTE III

Módulos del ecosistema

Esta parte describe en detalle los módulos opcionales que rodean al núcleo: las capacidades `batteries-included` que permiten construir aplicaciones de producción sin tener que ensamblar cada componente desde primitivas, y los subsistemas más ambiciosos (plugins WASI, despliegue, integración con IA, migración, móvil) que diferencian a Anti-Gravital de los frameworks Rust de bajo nivel disponibles hoy.

8. Módulos `batteries-included`

Los módulos estándar del ecosistema cubren las preocupaciones que aparecen en prácticamente toda aplicación de negocio. La política del proyecto es que cualquier capacidad requerida por más del setenta por ciento de los servicios de producción imaginables debe estar empaquetada como crate oficial mantenido. Esto evita la fragmentación que sufren ecosistemas más jóvenes, donde cada equipo termina escribiendo su propia capa de autenticación o su propia integración de observabilidad.

8.1 `ag-auth` — autenticación y autorización

El módulo `ag-auth` ofrece una implementación de autenticación moderna que prioriza la seguridad y la conveniencia operativa. WebAuthn es ciudadano de primera clase: registro, login y recuperación `passkey-first`, con fallback a métodos clásicos solo cuando el cliente lo solicita explícitamente. JWT firmado con Ed25519 reemplaza el viejo HS256/RS256 por una clave más corta y verificación más rápida. El módulo incluye OAuth2 con los proveedores comunes (Google, GitHub, Apple, Microsoft) preconfigurados, RBAC declarativo desde el schema `.ag`, rate limiting por usuario y por IP, y detección de credentials stuffing mediante un bloom filter de contraseñas comprometidas conocidas.

8.2 `ag-data` — capa de persistencia

`ag-data` se construye sobre `sqlx` y conserva su característica más valiosa: la verificación en tiempo de compilación de las consultas SQL contra el esquema real de la base de datos. Sobre esa base, añade un ORM tipado generado desde `schema.ag`, migraciones versionadas, soporte para PostgreSQL, MySQL y SQLite,

transacciones declarativas, paginación cursor-based estandarizada y soft deletes opcionales. La política del módulo es no esconder SQL del desarrollador: el ORM es una conveniencia para casos comunes, y el escape hatch a SQL crudo verificado en compile-time está siempre disponible.

8.3 ag-realtime — tiempo real bidireccional

El módulo `ag-realtime` integra WebSocket, Server-Sent Events y un broker de mensajes pub/sub interno basado en NATS embebido. La opción embebida permite que aplicaciones pequeñas funcionen sin desplegar un cluster NATS externo; la misma API soporta NATS clusterizado para producción a escala. Los canales se declaran desde el schema `.ag` con tipado fuerte: un evento mal tipado es un error de compilación, no un bug de runtime.

8.4 ag-cache — caché multinivel

`ag-cache` ofrece dos backends intercambiables: `moka` para caché en proceso de baja latencia, y Redis para caché compartido entre instancias. El módulo añade invalidación por evento — un cambio en un modelo emite eventos que invalidan entradas dependientes de manera automática — y políticas declarativas de TTL desde el schema. Esto elimina la clase de bug de caché obsoleto causado por invalidaciones manuales olvidadas.

8.5 ag-storage — almacenamiento de objetos

Abstracción sobre S3, MinIO y filesystem local con la misma API. `ag-storage` incluye URLs firmadas con expiración configurable, multipart uploads, validación de tipo MIME en el ingreso, y procesamiento de imágenes opcional (redimensionado, conversión a WebP/AVIF, generación de thumbnails) ejecutado dentro del propio proceso Rust mediante `image`. La integración con el resto del ecosistema permite que un campo `File` en el schema genere automáticamente endpoint de upload, validación, almacenamiento y URL firmada de descarga.

8.6 ag-observe — observabilidad

El módulo `ag-observe` empaqueta tres pilares: trazas distribuidas mediante OpenTelemetry y el ecosistema `tracing` de Rust, métricas en formato Prometheus, y logs estructurados en JSON. La instrumentación es automática: cada handler generado desde el DSL produce un span con los atributos correctos, cada query SQL se traza, cada llamada a plugin WASI se mide. Los dashboards Grafana oficiales se publican como parte del módulo y se importan con un comando de la CLI.

8.7 ag-ui — renderizado del lado del servidor

`ag-ui` es el módulo opcional para aplicaciones que prefieren el modelo SSR clásico, con plantillas `askama` verificadas en compile time e integración nativa con HTMX para interactividad sin un framework JavaScript pesado. Este módulo no compite con React, Vue ni Next.js: cubre el caso de uso de aplicaciones internas, dashboards administrativos y proyectos pequeños donde un stack JS completo es excesivo. Para aplicaciones SPA o SSR ricas, el patrón recomendado sigue siendo Anti-Gravital como backend con cliente TypeScript generado, consumido por un frontend Next.js o equivalente.

9. Sistema de plugins WASI

El sistema de plugins es la respuesta a una pregunta legítima: ¿cómo se permite que terceros extiendan Anti-Gravital sin obligar a todo el ecosistema a escribir Rust? La respuesta es WebAssembly System Interface, ejecutado sobre `wasmtime`, con un contrato de ABI versionado y un modelo de permisos explícito.

9.1 Por qué WASI y no shared libraries

Las shared libraries cargadas con `dlopen` comparten el espacio de memoria del proceso anfitrión. Un bug en la librería puede corromper el estado del runtime; un comportamiento malicioso puede leer secretos en memoria, abrir sockets arbitrarios o tocar el filesystem sin restricciones. WASI invierte ese modelo: el plugin se ejecuta en una sandbox de WebAssembly con permisos explícitos otorgados por el host. Sin permiso de red, el plugin no puede abrir un socket; sin permiso de filesystem, no puede leer archivos. El aislamiento es estructural, no contractual.

9.2 ABI del plugin

El contrato entre Anti-Gravital y un plugin WASI se define en un archivo `plugin.wit` usando la especificación WebAssembly Interface Types. Esto permite escribir plugins en Rust, Go (con TinyGo), C/C++, Zig, Python (vía componentize-py) o cualquier lenguaje que se compile a un componente WASI. El ABI declara las funciones que el plugin expone (`on_request`, `on_response`, `on_event`, etc.) y las funciones del host que el plugin puede llamar (`http_get`, `kv_set`, `log_info`, etc.).

9.3 Manifest del plugin

```
# plugin.toml – declaración explícita de capacidades
[plugin]
name      = "image-watermark"
version   = "1.2.0"
author    = "Comunidad Anti-Gravital"
abi-version = "1"

[capabilities]
http-outbound = ["api.cloudinary.com"]    # solo este dominio
filesystem    = { read = [], write = [] } # nada
env            = ["WATERMARK_KEY"]        # solo esta variable

[hooks]
on-upload = "process_image"
```

9.4 Ciclo de vida del plugin

El ciclo de vida está definido con precisión para evitar comportamientos sorpresivos. En el arranque del servidor, el plugin se carga, se verifica su firma criptográfica (si la política del proyecto lo exige), se instancia su módulo WASI y se llama a `init()` con la configuración. Durante la operación, los hooks declarados en el manifest se invocan en los puntos del request lifecycle correspondientes. En el apagado ordenado, se invoca `shutdown()` con un timeout configurable; si el plugin no termina dentro del timeout, el host fuerza la terminación de la instancia WASI sin afectar al proceso anfitrión.

10. ag-cloud: despliegue simplificado

El módulo `ag-cloud` es deliberadamente modesto en alcance. No es un reemplazo de Kubernetes, no es un competidor de Terraform, no es una plataforma multi-cloud completa. Es una capa de orquestación que cubre el rango de despliegues que va desde Docker Compose en una VPS hasta Fly.io en producción, con generación de Dockerfile, configuración de proxy reverso, gestión de secretos y health checks.

10.1 Cuatro targets de despliegue

El primer target es **docker-compose**, que produce un `docker-compose.yml` con el servicio Anti-Gravital, PostgreSQL, Redis si `ag-cache` está activo, NATS si `ag-realtime` está activo, y un reverse proxy Caddy con HTTPS automático. Este target es ideal para desarrollo, demos y despliegues pequeños en una sola máquina.

El segundo target es **fly**, que genera un `fly.toml` con la configuración de máquinas, regiones, escalado y secretos para desplegar en Fly.io. El módulo aprovecha la afinidad natural entre Fly.io y aplicaciones Rust nativas: cold starts de menos de un segundo, máquinas pequeñas a costo bajo, despliegue multirregión sencillo.

El tercer target es **railway**, que produce el `railway.json` y el Dockerfile compatible con la plataforma Railway. La filosofía de Railway de empaquetar todo en una experiencia tipo Heroku-moderno encaja bien con el modelo de Anti-Gravital de un único binario sin runtime externo.

El cuarto target es **k8s**, que genera manifiestos Kubernetes (Deployment, Service, Ingress, ConfigMap, HorizontalPodAutoscaler) listos para aplicar en cualquier cluster. Este target reconoce que para cargas más grandes la respuesta correcta es Kubernetes, y se limita a producir manifiestos buenos por defecto que el equipo puede ajustar.

10.2 Lo que ag-cloud no hace

El módulo no intenta gestionar infraestructura compleja, no orquesta múltiples servicios independientes, no provee un dashboard de monitorización propio, no compite con plataformas SaaS de PaaS. Para esas necesidades, la combinación correcta es Terraform o Pulumi para la infraestructura, ag-cloud para el binario de la aplicación, y Grafana o Datadog para la observabilidad.

11. ag-ai y Knowledge Graph

La integración con inteligencia artificial es probablemente el módulo donde el v3.0 prometía más de lo que podía sostener. La v4.0 la reduce a tres capacidades concretas y verificables, sin retórica sobre agentes autónomos ni promesas de generación end-to-end.

11.1 Documentación auto-generada

La primera capacidad es la generación automática de documentación de arquitectura desde el código y los archivos `.ag`. El módulo construye un grafo de conocimiento — el **AG Knowledge Graph** — que indexa modelos, endpoints, relaciones, dependencias entre módulos, queries SQL y eventos. Sobre ese grafo, un modelo de lenguaje produce descripciones legibles, diagramas Mermaid actualizados, y respuestas a preguntas tipo «¿qué endpoints consultan la tabla `orders`?» o «¿qué eventos emite el módulo de facturación?».

11.2 Sugerencias de schema

La segunda capacidad es la sugerencia asistida al editar archivos `.ag`. Cuando el desarrollador describe un modelo en lenguaje natural en un comentario, el LSP del DSL puede ofrecer una sugerencia estructurada que el desarrollador acepta o modifica. Esto no genera código a ciegas: produce una propuesta de schema que pasa por el resto del pipeline normal (compilador, validación, code review).

11.3 Análisis y diagnóstico

La tercera capacidad es el análisis del Knowledge Graph para identificar problemas estructurales: endpoints sin auth declarado, modelos sin migraciones aplicadas, queries con planes de ejecución sospechosos, dependencias circulares entre módulos. El módulo no «corrige» el problema; lo reporta con un mensaje accionable y, cuando es posible, una propuesta de cambio.

12. ag-migrate: importadores de migración

Una de las decisiones estratégicas más importantes derivadas del análisis crítico fue elevar la migración desde stacks legacy a la categoría de módulo de primera clase. La adopción de Anti-Gravital se acelera enormemente cuando un equipo puede importar su contrato existente — sea OpenAPI, Prisma, Django ORM o lo que tenga — y obtener un proyecto Anti-Gravital funcional en minutos en lugar de reescribir manualmente cientos de modelos.

12.1 Importadores oficiales

Los seis importadores oficiales son: **OpenAPI 3.x**, que produce un schema `.ag` con endpoints, modelos y validaciones desde una especificación YAML/JSON; **Prisma schema**, que convierte modelos Prisma y sus relaciones; **Django ORM**, que parsea `models.py` y produce el schema equivalente, incluyendo la mayoría de las relaciones y validadores comunes; **FastAPI/Pydantic**, que extrae modelos Pydantic y rutas FastAPI mediante introspección AST; **Sequelize**, que importa modelos JavaScript del ORM más usado en el ecosistema Node.js; y **GraphQL SDL**, que produce schema `.ag` desde un Schema Definition Language de GraphQL.

12.2 Modelo de importación incremental

Los importadores no son herramientas de un solo uso. Producen un schema base que el equipo refina, y soportan re-importación incremental: cuando el esquema fuente cambia, el importador genera un diff que el equipo aplica selectivamente. Esto permite migraciones graduales donde Anti-Gravital convive con el stack legacy durante meses, sirviendo nuevos endpoints mientras el equipo migra los antiguos a su propio ritmo.

13. ag-mobile: puente con Flutter

El módulo `ag-mobile` implementa la estrategia explícita de tratar a Flutter como un *target de integración*, no como un competidor. La tesis es que Anti-Gravital debe convertirse en el backend nativo ideal para aplicaciones Flutter, ofreciendo una experiencia de desarrollo que ningún stack JavaScript puede igualar.

13.1 Generación de SDK Dart

Desde el mismo `schema.ag` que produce el cliente TypeScript, el módulo genera un SDK Dart con modelos tipados, cliente HTTP, soporte de autenticación y wrappers de WebSocket/SSE. El SDK es idiomático para Flutter: usa `freezed` para inmutabilidad, `dio` como cliente HTTP por defecto (configurable), y se integra con `riverpod` y `bloc` para gestión de estado.

13.2 Autenticación nativa

El SDK Dart incluye autenticación nativa que aprovecha las capacidades de la plataforma móvil: Face ID y Touch ID en iOS, BiometricPrompt en Android, almacenamiento seguro de tokens en Keychain/Keystore, y soporte de passkeys cuando la plataforma lo permite. La experiencia de login en una app Flutter consumiendo Anti-Gravital es comparable a las apps nativas mejor diseñadas del mercado.

13.3 Offline sync

El componente más ambicioso del módulo es el sync offline-first. El SDK Dart mantiene una base de datos local SQLite — espejo parcial del esquema del servidor — y sincroniza con el backend usando un protocolo CRDT-like documentado. Los conflictos se resuelven con políticas declaradas en el schema `.ag`: last-write-wins, server-wins, custom-merge. Esta capacidad llega en una fase tardía del roadmap precisamente porque su corrección bajo todos los escenarios es difícil y requiere validación extensa antes de declararse estable.

PARTE IV

Calidad y operación

Esta parte cubre las preocupaciones transversales que determinan si un framework puede operarse responsablemente en producción: observabilidad, modelo de seguridad y objetivos de rendimiento con su metodología de validación.

14. Observabilidad

La observabilidad en Anti-Gravital no es una capa añadida después; está construida en el tejido del runtime desde el primer commit. Cada handler generado por `ag generate` produce un span de tracing con atributos coherentes, cada query SQL ejecutada por `ag-data` se instrumenta automáticamente, y cada llamada a un plugin WASI registra su duración y resultado. El desarrollador no escribe instrumentación: la recibe gratis del compilador del DSL.

14.1 Los tres pilares

El módulo `ag-observe` implementa los tres pilares clásicos de la observabilidad con tecnologías estándar de la industria. Las trazas distribuidas se exportan en formato OpenTelemetry (OTLP) a cualquier backend compatible, ya sea Jaeger, Tempo, Honeycomb o Datadog. Las métricas se exponen en formato Prometheus, con los counters, gauges e histogramas habituales para latencia, throughput, tasas de error y saturación de recursos. Los logs son JSON estructurados con campos de correlación que permiten saltar de un log a su traza y de una traza a sus logs sin esfuerzo manual.

14.2 Dashboards oficiales

El proyecto publica dashboards Grafana oficiales para los casos comunes: un dashboard de aplicación con latencia p50/p95/p99 por endpoint, un dashboard de base de datos con queries lentas y planes de ejecución problemáticos, un dashboard de runtime con uso de memoria, threads Tokio activos y backpressure. Estos dashboards se importan con `ag observe install-dashboards`, lo cual instala los archivos JSON en una instancia Grafana configurada.

14.3 Diagnóstico de incidentes

Cuando algo falla en producción, la pregunta crítica es: ¿cuánto tarda el equipo en entender qué pasó? El diseño de observabilidad de Anti-Gravital prioriza esta pregunta. Cada error en runtime incluye automáticamente el trace ID en el response, los logs estructurados se correlacionan con la traza distribuida, y el dashboard de aplicación expone los errores recientes con un click directo al span correspondiente. El objetivo de diseño es que el tiempo medio para identificar la causa raíz de un incidente baje al menos un orden de magnitud comparado con un stack JavaScript o Python equivalente sin instrumentación dedicada.

15. Modelo de seguridad

El modelo de seguridad de Anti-Gravital se construye sobre tres principios que se aplican en cada decisión arquitectónica: **seguridad por construcción**, **defensa en profundidad** y **mínimo privilegio**.

15.1 Seguridad por construcción

Rust elimina por construcción clases enteras de vulnerabilidades que han plagado a la industria durante décadas: use-after-free, double-free, buffer overflows, data races. Estas vulnerabilidades no son posibles en código Rust seguro, y el código `unsafe` del proyecto se mantiene auditado, justificado y minimizado. La política del proyecto es que cualquier bloque `unsafe` en el código del ecosistema debe estar acompañado de un comentario que documente la invariante que el bloque mantiene y por qué la abstracción más simple no era posible.

15.2 Defensa en profundidad

The Shield es la primera línea de defensa: TLS termina la conexión, el rate limiter rechaza el abuso volumétrico, la autenticación verifica la identidad, la autorización aplica RBAC, y la validación contra el schema garantiza que los datos del request son estructuralmente correctos antes de llegar al código de negocio. Cada capa puede rechazar el request por su propia razón, y cada capa es independiente: una falla en la lógica de RBAC no compromete la validación de TLS.

15.3 Mínimo privilegio

El sistema de plugins WASI aplica este principio con rigor: un plugin solo recibe los permisos que su manifest declara explícitamente. Sin permiso de red, no puede abrir un socket. Sin permiso de filesystem, no puede leer archivos. Sin acceso a variables de entorno, no puede leer secretos. La granularidad permite limitar la red a dominios específicos: un plugin que llama a una API externa declara exactamente qué dominios necesita, y el host bloquea cualquier intento de conexión a destinos no declarados.

15.4 Política de divulgación responsable

El proyecto mantiene una política formal de divulgación de vulnerabilidades documentada en `SECURITY.md`. Los reportes se reciben por canal cifrado, se confirma la recepción en un plazo de 48 horas, se evalúa la severidad usando CVSS 3.1, y se acuerda con el reportante un cronograma de divulgación coordinada. El equipo de seguridad del proyecto se compromete a publicar parches y advisories CVE para las vulnerabilidades confirmadas, con créditos a los reportantes que opten por la divulgación pública.

16. Objetivos de rendimiento

Esta sección requiere un tratamiento cuidadoso. Las afirmaciones de rendimiento en marketing técnico suelen ser optimistas, no reproducibles, o ambas cosas. El proyecto adopta la posición opuesta: los números que se publican son **objetivos de diseño** verificables mediante un harness de benchmarks público, ejecutado en hardware estandarizado, con código de medición disponible para que cualquiera reproduzca los resultados.

16.1 Objetivos de diseño

Métrica	Objetivo de diseño	Condición
Latencia p99 (handler trivial)	< 1 ms	en hardware de referencia, sin carga concurrente significativa
Latencia p99 (handler con query SQL simple)	< 5 ms	PostgreSQL local en la misma máquina, índice presente
Throughput sostenido	$\geq 100\,000$ req/s	handler trivial, en hardware de referencia, sin saturación de CPU
Memoria base por instancia	< 50 MB	incluyendo runtime Tokio, sin cache cargado
Tiempo de arranque en frío	< 100 ms	desde ejecución del binario hasta primer request servido
Tamaño del binario stripped	< 30 MB	compilación release con LTO, sin debug symbols
Tiempo de compilación incremental	< 5 s	cambio aislado en un handler, proyecto mediano

16.2 Metodología de validación

Cada objetivo de la tabla anterior tiene un benchmark asociado en el repositorio `anti-gravital/benchmarks`. El harness es público, está versionado, ejecuta en un runner CI dedicado con hardware estable, y publica resultados históricos en un dashboard accesible. Cuando un commit hace regresar una métrica más allá de su umbral de tolerancia, el CI bloquea el merge y exige un análisis de la causa.

16.3 Lo que estos números no significan

Es importante ser explícito sobre lo que estos objetivos *no* significan. No son una afirmación de que Anti-Gravital sea «más rápido que X». Cada framework tiene un perfil de carga donde se desempeña mejor; las comparaciones entre frameworks dependen profundamente del benchmark elegido. Los números publicados son una caracterización honesta del comportamiento del runtime bajo las condiciones declaradas, no una proclamación competitiva.

PARTE V

El proyecto

Esta parte trata los aspectos extra-técnicos que determinan si Anti-Gravital se convierte en un proyecto vivo y adoptado o en un repositorio archivado: gobernanza open source, análisis de riesgos, hoja de ruta de desarrollo y estrategia go-to-market.

17. Modelo de gobernanza Open Source

Un framework de la ambición de Anti-Gravital no puede depender indefinidamente de un único mantenedor. El modelo de gobernanza se diseña desde el día uno para transitar de manera ordenada desde un liderazgo benevolente inicial a un comité técnico distribuido cuando el proyecto alcance la masa crítica que lo justifique.

17.1 Fase BDFL (v0.0 → v1.0)

Durante el periodo de pre-lanzamiento, el proyecto opera bajo un modelo BDFL (*Benevolent Dictator For Life*) con Angel Nereira como mantenedor principal. Este modelo es pragmático para proyectos en fase de definición arquitectónica: la velocidad de iteración y la coherencia de visión son más valiosas que el consenso distribuido. El BDFL se compromete a documentar las decisiones arquitectónicas mediante ADR (*Architectural Decision Records*) firmadas y publicadas, de modo que la racionalidad de cada decisión sea inspeccionable por la comunidad y por mantenedores futuros.

17.2 Transición a comité técnico (post v1.0)

Una vez que el proyecto alcance v1.0 estable y se haya consolidado una comunidad de mantenedores activos (al menos cinco mantenedores distintos con al menos un año de contribuciones significativas), se constituye un **Anti-Gravital Technical Committee** de entre cinco y siete miembros. El BDFL conserva un voto y un derecho de veto limitado en cuestiones de coherencia arquitectónica fundamental, pero las decisiones técnicas operativas pasan al comité.

17.3 Licencia y compromiso de apertura

La totalidad del ecosistema se publica bajo licencia Apache 2.0. El proyecto se compromete formalmente — y este compromiso queda incrustado en el `README.md` del repositorio principal — a no crear nunca una versión Enterprise cerrada, a no reservar features para clientes pagos, a no relicenciar el código bajo términos más

restrictivos, y a no aceptar contribuciones que requieran CLA exclusivos que permitan relicenciamiento futuro. El modelo de financiación del proyecto, cuando exista, se basa en consultoría, soporte profesional, hosting gestionado opcional y patrocinios corporativos, no en una versión de pago del software.

18. Análisis de riesgos

Esta sección documenta los riesgos identificados para el proyecto y las mitigaciones planificadas. La honestidad sobre los riesgos es una propiedad esencial de un blueprint creíble: ocultarlos no los elimina, solo retrasa el momento en el que se manifiestan.

18.1 Riesgo de complejidad acumulada

El riesgo más serio del proyecto es que la ambición del alcance produzca una codebase demasiado compleja para mantener con los recursos disponibles. La mitigación es la separación estricta en crates independientes con propietarios responsables, la regla de que cada módulo opcional puede archivar sin afectar al núcleo, y la disciplina de las puertas de verificación de la hoja de ruta que impiden avanzar a una fase nueva sin haber consolidado la anterior.

18.2 Riesgo de adopción lenta

Rust, a pesar de su crecimiento, todavía es un lenguaje minoritario comparado con JavaScript, Python o Java. La barrera de entrada de Rust es real y puede limitar la velocidad de adopción del framework. La mitigación es triple: el DSL reduce la cantidad de código Rust que el desarrollador debe escribir directamente, los importadores de `ag-migrate` permiten migración gradual desde stacks familiares, y la documentación bilingüe español/inglés desde el día cero abre un mercado latinoamericano que ningún framework Rust prioriza hoy.

18.3 Riesgo de divergencia del ecosistema Rust

El ecosistema Rust evoluciona rápido y las dependencias clave (Tokio, Axum, sqlx, wasmtime) pueden introducir breaking changes. La mitigación es contribuir activamente a esas dependencias, mantener pinning explícito de versiones en el workspace, y diseñar las APIs de Anti-Gravital con una capa de adaptación que aisle al usuario de los cambios upstream.

18.4 Riesgo de competencia

El espacio de frameworks Rust no es vacío. Loco, Pavex y otros proyectos persiguen objetivos parcialmente solapados. La mitigación no es competir con marketing sino con producto: el alcance del ecosistema completo (DSL + módulos + plugins + cloud + AI + migrate + mobile) es estructuralmente más amplio que el de cualquier competidor actual, y la disciplina arquitectónica de mantener cada pieza pequeña y bien especificada hace al ecosistema defendible.

18.5 Riesgo de captura corporativa

Un proyecto open source exitoso es siempre vulnerable a la captura por intereses corporativos que intenten orientarlo a sus necesidades específicas. La mitigación es la gobernanza distribuida post-v1.0, la licencia Apache 2.0 que garantiza el derecho a fork, y la transparencia obligatoria de las decisiones arquitectónicas mediante ADRs públicos.

19. Hoja de ruta y puertas de verificación

El desarrollo del ecosistema se organiza en once fases secuenciales numeradas de 0 a 10. Cada fase tiene criterios de entrada, una lista cerrada de entregables, criterios de salida que actúan como puertas bloqueantes, y un análisis de riesgos específico. No se avanza a la fase siguiente hasta que todos los criterios de salida de la fase actual están verificados y firmados.

DISCIPLINA DE LAS PUERTAS

La regla más importante del proceso es la siguiente: **no se avanza a la fase siguiente hasta que todos los criterios de salida de la fase actual estén cumplidos**. Las puertas son bloqueantes por diseño. Saltarse una puerta porque «el siguiente sprint es importante» es la forma más común en que los proyectos open source ambiciosos se desintegran.

19.1 Resumen del cronograma

Fase	Nombre	Duración	Hito
0	Fundamentos y gobernanza	1–2 meses	Repositorio público, CI verde
1	Shield MVP	2–3 meses	TLS+JWT+RBAC en producción de prueba
2	Core MVP + roundtrip	2 meses	Hello world end-to-end
3	Anti-DSL alpha (v0.1–v0.4)	3 meses	Schema-first funcional
4	Módulos estándar	3 meses	Auth, data, realtime, cache, storage
5	ag-cloud y v0.5 BETA	2 meses	Hito v0.5 BETA público
6	ag-ai y Knowledge Graph	2 meses	Documentación auto-generada
7	ag-migrate (importadores)	2 meses	6 importadores oficiales
8	ag-mobile (Flutter)	2 meses	SDK Dart con offline sync
9	Plugins WASI	2 meses	ABI v1 estable
10	Hardening y v1.0 estable	3 meses	v1.0 GA

El total de duración estimada es de 24 a 28 meses desde el inicio de la fase 0 hasta el hito v1.0 GA. El cronograma es deliberadamente conservador: cada fase incluye márgenes para imprevistos, y la disciplina de las puertas hace preferible estirar una fase a comprometer su completitud.

19.2 Fase 0 — Fundamentos y gobernanza

FASE 0 Fundamentos y gobernanza

1–2 meses

Entregables: repositorio público bajo Apache 2.0, workspace Cargo configurado con los 15 crates vacíos, CI con GitHub Actions que ejecuta `cargo check`, `cargo test`, `cargo clippy` y `cargo fmt --check` en cada PR, documentos de gobernanza completos (`README.md`, `CONTRIBUTING.md`, `CODE_OF_CONDUCT.md`, `SECURITY.md`, `GOVERNANCE.md`), proceso ADR documentado con plantilla, dominio `anti-gravital.org` registrado y sitio mínimo publicado.

PUERTA DE SALIDA — TODOS BLOQUEANTES

- CI verde sobre el workspace completo
- Documentos de gobernanza revisados y publicados
- Al menos un ADR inicial publicado (decisión de licencia)
- Página de proyecto pública con CTA «star» y enlace al repositorio

19.3 Fase 1 — Shield MVP

FASE 1 Shield MVP

2–3 meses

Entregables: implementación de `ag-core::shield` con las capas TLS (rustls), JWT Ed25519, rate limiting (governor), CORS y validación estructural; suite de tests unitarios con cobertura mínima del 85 %; benchmarks iniciales del Shield aislado; documentación de cada capa con ejemplos de uso; ADR publicado por cada decisión arquitectónica significativa.

PUERTA DE SALIDA — TODOS BLOQUEANTES

- Cobertura de tests $\geq 85\%$ en `ag-core::shield`
- Benchmarks del Shield aislado documentados y reproducibles
- Auditoría de seguridad interna del código `unsafe` presente (debe ser cero o estar plenamente justificado)
- Demo público con servidor TLS+JWT funcionando contra cliente real

19.4 Fase 2 — Core MVP y roundtrip

FASE 2 Core MVP + roundtrip

2 meses

Entregables: implementación de `ag-core::core` con router Axum, extractores tipados, error types unificados; primera versión funcional de `ag-cli` con los comandos `new`, `build` y `dev`; template oficial de proyecto que produce un binario hello-world funcional; documentación tutorial paso a paso.

PUERTA DE SALIDA — TODOS BLOQUEANTES

- `ag new my-app && cd my-app && ag dev` produce un servidor funcional
- Latencia p99 del handler trivial < 5 ms en hardware de referencia
- Tutorial getting-started revisado por al menos tres usuarios externos
- Tamaño del binario stripped < 30 MB

19.5 Fase 3 — Anti-DSL alpha

FASE 3 Anti-DSL alpha (v0.1 → v0.4)

3 meses

Entregables: implementación incremental del compilador del DSL en cuatro sub-fases (modelos, endpoints, validaciones, relaciones); generación de modelos Rust con serde; generación de handlers stub; generación de migraciones SQL básicas; generación de cliente TypeScript inicial; LSP mínimo con autocompletado e ir-a-definición; `ag generate` integrado en la CLI.

PUERTA DE SALIDA — TODOS BLOQUEANTES

- Gramática v0.4 documentada con ejemplos en la web del proyecto
- Suite de tests del compilador con cobertura $\geq 80\%$
- LSP funcional probado en VS Code y Neovim
- Demo público de schema con relaciones generando código funcional

19.6 Fase 4 — Módulos estándar

FASE 4 Módulos estándar

3 meses

Entregables: publicación de `ag-auth` con WebAuthn y JWT; `ag-data` con sqlx PostgreSQL/MySQL/SQLite y migraciones; `ag-realtime` con WebSocket, SSE y NATS embebido; `ag-cache` con moka y Redis; `ag-storage` con S3, MinIO y filesystem; `ag-observe` con OpenTelemetry, Prometheus y dashboards Grafana oficiales; integración de cada módulo con el DSL.

PUERTA DE SALIDA — TODOS BLOQUEANTES

- Cada módulo publicado en crates.io con versión > 0.4
- Cobertura de tests ≥ 75 % en cada módulo
- Aplicación de referencia que usa los seis módulos en producción de prueba
- Dashboards Grafana oficiales importables con un comando

19.7 Fase 5 — ag-cloud y v0.5 BETA

FASE 5 ag-cloud y v0.5 BETA público

2 meses

Entregables: implementación de `ag-cloud` con los cuatro targets (docker-compose, fly, railway, k8s); generación de Dockerfile multi-stage optimizado; comandos `ag deploy` con feedback estructurado; documentación de cada target con ejemplo end-to-end; **lanzamiento público v0.5 BETA** con anuncio formal, post de blog, video demo, presencia en redes técnicas.

PUERTA DE SALIDA — TODOS BLOQUEANTES

- Despliegue verificado en los cuatro targets
- Post de lanzamiento publicado con ≥ 500 estrellas iniciales
- Al menos cinco contribuyentes externos con PRs mergeados
- Canales de comunidad (Discord/Matrix, foro) operativos con moderación

19.8 Fase 6 — ag-ai y Knowledge Graph

FASE 6 ag-ai y Knowledge Graph

2 meses

Entregables: implementación del Knowledge Graph indexando modelos, endpoints, relaciones, queries y eventos; generador de documentación arquitectónica con diagramas Mermaid; sugerencias de schema en el LSP; análisis estático del grafo para detección de problemas estructurales; integración opcional con proveedores de LLM externos para casos donde el usuario lo autoriza explícitamente.

PUERTA DE SALIDA — TODOS BLOQUEANTES

- Knowledge Graph reproducible mediante un comando para cualquier proyecto Anti-Gravital
- Documentación arquitectónica generada para la app de referencia
- Análisis estático detecta al menos cinco clases de problemas estructurales documentados
- Política de privacidad y uso de datos clara para la integración opcional con LLMs

19.9 Fase 7 — ag-migrate (importadores)

FASE 7 ag-migrate: importadores

2 meses

Entregables: implementación de los seis importadores oficiales (OpenAPI, Prisma, Django ORM, FastAPI/Pydantic, Sequelize, GraphQL SDL); soporte de importación incremental con generación de diffs; documentación de cada importador con casos de uso reales; aplicación demo migrada desde cada stack legacy a Anti-Gravital.

PUERTA DE SALIDA — TODOS BLOQUEANTES

- Cada importador validado contra al menos diez proyectos open source reales del stack original
- Pipeline de import incremental funciona con regeneración del schema
- Documentación de cada importador con limitaciones conocidas explícitas
- Video demo de cada importador en condiciones reales

19.10 Fase 8 — ag-mobile (Flutter)

FASE 8 ag-mobile: Flutter bridge

2 meses

Entregables: generador de SDK Dart desde `schema.ag`; integración nativa de WebAuthn/biometría en iOS y Android; componentes de autenticación pre-construidos para Flutter; sincronización offline-first con SQLite local y resolución de conflictos declarativa; aplicación móvil de referencia publicable en TestFlight y Google Play.

PUERTA DE SALIDA — TODOS BLOQUEANTES

- SDK Dart publicado en pub.dev con versión > 0.4
- App Flutter de referencia funcional en iOS y Android
- Offline sync validado con suite de tests que cubre escenarios de conflicto documentados
- Documentación específica para desarrolladores Flutter

19.11 Fase 9 — Plugins WASI

FASE 9 Plugins WASI

2 meses

Entregables: implementación de `ag-wasm-host` sobre wasmtime; ABI versionada documentada en `plugin.wit`; sistema de manifest con declaración de capacidades; ciclo de vida con init/hooks/shutdown; ejemplos de plugins en Rust, Go (TinyGo) y Python (componentize-py); registro público de plugins de la comunidad.

PUERTA DE SALIDA — TODOS BLOQUEANTES

- ABI v1 documentada y declarada estable
- Al menos cinco plugins de ejemplo en lenguajes diferentes
- Auditoría de seguridad del sistema de permisos por revisor externo
- Registro público de plugins con políticas de moderación

19.12 Fase 10 — Hardening y v1.0 estable

FASE 10 Hardening y v1.0 estable

3 meses

Entregables: auditoría de seguridad externa contratada con firma reputada; programa de bug bounty público; congelación de la gramática del DSL en v1.0 con proceso RFC formal para cambios futuros; documentación completa revisada en español e inglés; al menos diez casos de estudio de producción documentados; **lanzamiento v1.0 GA** con anuncio formal y constitución del Anti-Gravital Technical Committee.

PUERTA DE SALIDA — TODOS BLOQUEANTES

- Auditoría externa de seguridad completada y reporte público
- Cero issues críticos abiertos en el tracker
- Documentación completa en español e inglés revisada por al menos dos revisores externos
- Diez casos de estudio de producción documentados con métricas reales
- Technical Committee constituido y operativo

20. Estrategia go-to-market

La adopción de un framework no se gana solo con calidad técnica. Esta sección documenta la estrategia de comunicación, posicionamiento y crecimiento de comunidad que acompaña al desarrollo técnico durante las once fases.

20.1 Tres fases de adopción

La primera fase, durante el desarrollo pre-v0.5, prioriza la construcción silenciosa sobre la visibilidad. El proyecto opera con un blog técnico de baja frecuencia que documenta decisiones arquitectónicas, un canal Discord/Matrix público pero pequeño, y una presencia en redes técnicas centrada en compartir aprendizajes, no en marketing de producto. El objetivo de esta fase no es captar usuarios; es construir credibilidad arquitectónica con la audiencia más exigente.

La segunda fase, alrededor del lanzamiento v0.5 BETA, marca el inicio de la comunicación activa. Se publica un post de lanzamiento detallado con métricas iniciales reproducibles, se solicita feedback estructurado, se organizan presentaciones en conferencias técnicas (Rust Conf LATAM, Open Source Summit, conferencias regionales) y se inicia el programa formal de early adopters con soporte directo del equipo core para los primeros diez equipos que pongan Anti-Gravital en producción.

La tercera fase, post v1.0 GA, escala la estrategia: contenido educativo regular (libro «Anti-Gravital in Action» en español e inglés, cursos en línea oficiales, certificaciones de mantenedores), patrocinios corporativos formalizados a través de la entidad legal del proyecto, y expansión geográfica más allá de Latinoamérica con traducciones a inglés, portugués brasileño y francés como prioridad.

20.2 Diferenciadores comunicables

La estrategia de comunicación se construye sobre tres mensajes diferenciadores que cualquier desarrollador puede entender en menos de un minuto. Primero: *un binario, sin runtime externo, listo para producción*. Segundo: *un schema, todo el código generado, sin schema drift posible*. Tercero: *ecosistema completo desde el día uno: auth, data, realtime, observabilidad, despliegue*. Cada mensaje se respalda con una demo de menos de tres minutos y métricas reproducibles.

20.3 Política frente a la comparación adversarial

El proyecto adopta una política explícita de comunicación: no se publican comparaciones adversariales con frameworks específicos. Las comparaciones técnicas, cuando son necesarias, se presentan como análisis de trade-offs en condiciones declaradas, no como afirmaciones de superioridad. Esta política protege la credibilidad del proyecto a largo plazo y evita el desgaste reputacional que sufren los proyectos que se posicionan «contra» otros.

APÉNDICES

Material de referencia

Los apéndices reúnen material de referencia consultable de manera independiente: el glosario de términos técnicos usados a lo largo del documento, y la comparativa de mercado que sitúa a Anti-Gravital frente al resto del ecosistema actual de frameworks web.

Apéndice A — Glosario técnico

Las definiciones de este glosario reflejan el sentido específico que los términos toman dentro del proyecto Anti-Gravital. Cuando un término del glosario aparece en el cuerpo del documento, su uso es consistente con la definición aquí registrada.

Término	Definición
Anti-DSL	Lenguaje de definición de dominio del proyecto, archivos con extensión <code>.ag</code> , usado para describir modelos, endpoints, validaciones, errores y políticas como contrato único de la aplicación.
ABI	Application Binary Interface. En el contexto de plugins WASI, el contrato binario versionado entre el host Anti-Gravital y los módulos WebAssembly cargables.
ADR	Architectural Decision Record. Documento firmado que registra una decisión arquitectónica significativa con su contexto, las opciones evaluadas, la decisión tomada y sus consecuencias.
BDFL	Benevolent Dictator For Life. Modelo de gobernanza inicial del proyecto durante la fase pre-v1.0, con un mantenedor principal responsable de las decisiones de coherencia arquitectónica.
Cold start	Tiempo desde la ejecución del binario hasta el primer request servido. Métrica crítica en entornos serverless y para escalado dinámico de instancias.
CRDT	Conflict-free Replicated Data Type. Familia de estructuras de datos que permiten resolver de manera determinista los conflictos de sincronización entre réplicas, utilizada por el offline sync de <code>ag-mobile</code> .
DSL	Domain-Specific Language. Lenguaje diseñado para un dominio específico, en contraposición a un lenguaje de propósito general.
GC	Garbage Collector. Mecanismo automático de gestión de memoria presente en JVM, CLR, V8, CPython y otros runtimes gestionados. Ausente por construcción en Rust y, por tanto, en Anti-Gravital.
GIL	Global Interpreter Lock. Mecanismo de CPython que impide la ejecución concurrente real de bytecode Python en hilos del mismo proceso, principal limitante estructural del rendimiento CPU-bound de Python.
HNSW	Hierarchical Navigable Small World. Algoritmo de búsqueda aproximada de vecinos cercanos usado por <code>pgvector</code> para indexar embeddings vectoriales con alto rendimiento.
HTMX	Librería frontend que extiende HTML con atributos para realizar peticiones AJAX, WebSocket y SSE sin escribir JavaScript explícito. Soportada nativamente por <code>ag-ui</code> .
Knowledge Graph (AG)	Grafo de conocimiento del proyecto Anti-Gravital. Indexa modelos, endpoints, relaciones, queries y eventos para alimentar la documentación auto-generada y el análisis estructural.
LSP	Language Server Protocol. Protocolo estándar de comunicación entre editores e implementaciones de soporte a lenguajes. El LSP de Anti-DSL provee autocompletado, ir-a-definición y sugerencias asistidas.
OTLP	OpenTelemetry Protocol. Protocolo estándar de exportación de telemetría (trazas, métricas, logs) usado por <code>ag-observe</code> para integrarse con cualquier backend compatible.
RBAC	Role-Based Access Control. Modelo de autorización basado en roles, declarable desde el schema <code>.ag</code> mediante anotaciones de policy.

Término	Definición
RFC	Request for Comments. Proceso formal de propuesta y discusión pública de cambios significativos al ecosistema, obligatorio para modificaciones a la gramática del DSL post-v1.0.
RLS	Row-Level Security. Mecanismo de PostgreSQL para aplicar políticas de visibilidad y modificación a nivel de fila individual, soportado nativamente por <code>ag-data</code> .
Schema drift	Divergencia no intencional entre el contrato declarado de una API y su implementación efectiva. Eliminado por construcción en Anti-Gravital al generar todos los artefactos desde el mismo <code>schema.ag</code> .
Shield (The)	Capa A del runtime: pipeline de middleware Tower que aplica TLS, autenticación, rate limiting, RBAC y validación estructural antes de pasar el request al Core.
SSE	Server-Sent Events. Protocolo unidireccional servidor → cliente sobre HTTP, soportado por <code>ag-realtime</code> como alternativa ligera a WebSocket cuando la comunicación es de una sola vía.
SDL	Schema Definition Language. Sintaxis de definición de schemas, particularmente en el contexto de GraphQL. Uno de los formatos importables por <code>ag-migrate</code> .
WASI	WebAssembly System Interface. Estándar de interfaces para que módulos WebAssembly accedan a recursos del sistema operativo de manera controlada. Base del sistema de plugins de Anti-Gravital.
WebAuthn	Web Authentication. Estándar W3C para autenticación basada en credenciales criptográficas (passkeys, llaves de seguridad físicas, biometría de plataforma). Método primario de autenticación en <code>ag-auth</code> .

Apéndice B — Comparativa de mercado

La comparativa de la tabla siguiente posiciona a Anti-Gravital frente al resto del ecosistema actual sobre dimensiones específicas y verificables. La intención no es proclamar superioridad sino documentar las dimensiones donde el proyecto adopta posiciones diferenciadas, y declarar honestamente las dimensiones donde otros frameworks tienen ventajas legítimas (ecosistema maduro, base instalada amplia, talento disponible) que Anti-Gravital no puede compensar a corto plazo.

Dimensión	Anti-Gravital	Spring Boot	Django/FastAPI	Node.js	Axum solo
Lenguaje base	Rust	Java/Kotlin	Python	JavaScript	Rust
Runtime gestionado	No	Sí (JVM)	Sí (CPython)	Sí (V8)	No
Garbage Collector	No	Sí	Sí	Sí	No
Seguridad de memoria garantizada	Sí	Parcial	Parcial	Parcial	Sí
DSL schema-first oficial	Sí (Anti-DSL)	No	No	No	No
Generación de cliente tipado	Sí (TS, Dart)	Vía OpenAPI	Vía OpenAPI	Vía OpenAPI	Manual
Módulos batteries-included	Sí	Sí	Sí (Django)	Fragmentado	No
Sistema de plugins aislado	Sí (WASI)	No	No	No	No
Importadores de migración	Sí (6 oficiales)	No	No	No	No
Documentación bilingüe ES/EN	Sí desde día 0	Parcial	Parcial	Parcial	EN primario
Ecosistema maduro	En construcción	Sí	Sí	Sí	Parcial
Base instalada amplia	No	Sí	Sí	Sí	Creciente
Disponibilidad de talento	Baja	Alta	Alta	Muy alta	Media

Las tres últimas filas son honestas: Anti-Gravital es un proyecto naciente y carece, por definición, del ecosistema maduro, la base instalada y la disponibilidad de talento que poseen los frameworks con dos décadas de historia. Estas limitaciones son temporales y se compensan con la diferenciación arquitectónica que las primeras filas documentan, pero no deben ocultarse a quien evalúa adopción.

Anti-Gravital Framework

Blueprint v4.0 — Documento Maestro

Este documento consolida la arquitectura, los compromisos técnicos y la hoja de ruta del framework Anti-Gravital. Sustituye y deroga las versiones previas del blueprint. Su contenido se rige por la licencia Apache 2.0 y se publica con la misma libertad que el código del proyecto.

El proyecto invita a la comunidad técnica internacional a contribuir, criticar y construir sobre estas bases. La calidad final del ecosistema dependerá tanto del rigor de la implementación como del rigor del proceso de revisión externa al que este documento se somete deliberadamente.